

Reviewer Pack — Non-Commutative Rank of ACC^0 Circuits Bounded by Logarithmic Depth

Entry e50c336911a7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 19:25:53 UTC

Non-Commutative Rank of ACC^0 Circuits Bounded by Logarithmic Depth

Entry ID: e50c336911a7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 19:25:53 UTC

1. Conjecture

Field A (mathematical branch): Non-Commutative Algebra **Field B** (complexity object): ACC^0 Circuit Size

Statement:

For any ACC^0 circuit computing a function on n variables, the non-commutative rank of its gate algebra representation is $\Theta(\log n)$ if and only if the circuit depth is $\Theta(\log n)$.

Rationale (proposer's reasoning):

Non-commutative rank captures algebraic dependencies between gates, potentially exposing structural constraints in ACC^0 circuits. Logarithmic depth is a critical parameter for separating ACC^0 from NEXP, making this link both novel and theoretically grounded.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 52a65fa5bee9b799

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            factor = 1 / A[i][i]
            for j in range(n):
                A[i][j] *= factor
            for k in range(m):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def non_commutative_rank(A):
        m, n = len(A), len(A[0])
        rank = 0
        for i in range(m):
            if any(A[i][j] != 0 for j in range(n)):
                rank += 1
        return rank

    def generate_acc0_circuit(depth, n):
        circuit = []
        for _ in range(depth):
            gate = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(n)]
            circuit.append((gate, inputs))
        return circuit

    def evaluate_circuit(circuit):
        stack = []
        for gate, inputs in reversed(circuit):
            if gate == 'AND':
                result = all(inputs)
            elif gate == 'OR':
                result = any(inputs)
            stack.append(result)
```

```

    return stack.pop()

def matrix_representation(circuit, n):
    m = len(circuit)
    A = [[0] * (n + 1) for _ in range(m)]
    for i, (_, inputs) in enumerate(circuit):
        for j in range(n):
            A[i][j] = inputs[j]
        A[i][n] = evaluate_circuit(circuit[:i+1])
    return A

depth = random.randint(1, 10)
n = random.randint(5, 40)
circuit = generate_acc0_circuit(depth, n)
A = matrix_representation(circuit, n)
rank = non_commutative_rank(A)

expected_rank = math.log2(n) if depth > 0 else 0
conjecture_holds = abs(rank - expected_rank) < 1e-6

return {
    "metric_name": "non_commutative_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Depth {depth}, n={n}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"depth_vs_rank\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data or too many failures")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
on_commutative_rank', 'metric_value': 6, 'instances_tested': 1, 'conjecture_holds': False, 'countere
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 10, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 4, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 3, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 1, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 6, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 5, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 9, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 3, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 10, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 4, 'instances_tested': 1, 'conjecture
TRIAL: {'metric_name': 'non_commutative_rank', 'metric_value': 4, 'instances_tested': 1, 'conjecture
RESULT: INCONCLUSIVE insufficient data or too many failures
```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	108137
2	propose	ollama_remote	qwen3:8b	0	0	121931
3	preregistration	ollama_remote	qwen3:8b	0	0	24109
4	novelty	ollama_remote	qwen3:8b	0	0	20774
5	novelty	ollama_remote	qwen3:8b	0	0	15273
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10117
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9992
8	critic	ollama_remote	qwen3:8b	0	0	30039
9	judge	ollama_remote	qwen3:8b	0	0	21128

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 361500 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e50c336911a7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e50c336911a7.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e50c336911a7.tar.gz` (if generated)